

An Analysis of Momentum optimization

Momentum optimization

Momentum optimization -- Part 1

$$m_w(t) := \beta m_w(t-1) + (1 - \beta) \nabla L(w(t-1))$$
$$\{ \displaystyle m_w(t) := \beta m_w(t-1) + (1 - \beta) \nabla L(w(t-1)) \}$$

Momentum optimization -- Part 2

RMSProp has shown good adaptation of learning rate in different applications. RMSProp can be seen as a generalization of Rprop and is capable to work with mini-batches as well opposed to only full-batches.

Momentum optimization -- Part 3

Sign-based stochastic gradient descent Even though sign-based optimization goes back to the aforementioned Rprop, in 2018 researchers tried to simplify Adam by removing the magnitude of the stochastic gradient from being taken into account and only considering its sign. This results in a significantly lower communication cost of transferring gradients from workers to the parameter server. In this sense, it serves to better compress the gradient information, while having comparable convergence to standard SGD.

Momentum optimization -- Part 4

$$Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (w_1 + w_2 x_i - y_i)^2$$
$$\{ \displaystyle Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (w_1 + w_2 x_i - y_i)^2 \}$$

Momentum optimization -- Part 5

$\bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i$ When optimization is done, this averaged parameter vector takes the place of w .

Momentum optimization -- Part 6

where the parameter w which minimizes $Q(w)$ is to be estimated, η is a step size (sometimes called the learning rate in machine learning) and α is an exponential decay factor between 0 and 1 that determines the relative contribution of the current gradient and earlier gradients to the weight change. The name momentum stems from an analogy to momentum in physics: the weight vector w , thought of as a particle traveling through parameter space, incurs acceleration from the gradient of the loss ("force"). Unlike in classical

stochastic gradient descent, it tends to keep traveling in the same direction, preventing oscillations. Momentum has been used successfully by computer scientists in the training of artificial neural networks for several decades. The momentum method is closely related to underdamped Langevin dynamics, and may be combined with simulated annealing. In mid-1980s the method was modified by Yurii Nesterov to use the gradient predicted at the next point, and the resulting so-called Nesterov Accelerated Gradient was sometimes used in ML in the 2010s.

Momentum optimization -- Part 7

where i is uniformly sampled between 1 and n . Although theoretical convergence of this procedure happens under relatively mild assumptions, in practice the procedure can be quite unstable. In particular, when η is misspecified so that $I - \eta \nabla^2 Q$ has large absolute eigenvalues with high probability, the procedure may diverge numerically within a few iterations. In contrast, implicit stochastic gradient descent (shortened as ISGD) can be solved in closed-form as:

Momentum optimization -- Part 8

Implicit updates (ISGD) As mentioned earlier, classical stochastic gradient descent is generally sensitive to learning rate η . Fast convergence requires large learning rates but this may induce numerical instability. The problem can be largely solved by considering implicit updates whereby the stochastic gradient is evaluated at the next iterate rather than the current one:

Momentum optimization -- Part 9

The step size is denoted by η (sometimes called the learning rate in machine learning) and here $:=$ denotes the update of a variable in the algorithm. In many cases, the summand functions have a simple form that enables inexpensive evaluations of the sum-function and the sum gradient. For example, in statistics, one-parameter exponential families allow economical function-evaluations and gradient-evaluations. However, in other cases, evaluating the sum-gradient may require expensive evaluations of the gradients from all summand functions. When the training set is enormous and no simple formulas exist, evaluating the sums of gradients becomes very expensive, because evaluating the gradient requires evaluating all the summand functions' gradients. To economize on the computational cost at every iteration, stochastic gradient descent samples a subset of summand functions at every step. This is very effective in the case of large-scale machine learning problems.

An Analysis of Momentum optimization

Momentum optimization

Momentum optimization -- Part 10

Further proposals include the momentum method or the heavy ball method, which in ML context appeared in Rumelhart, Hinton and Williams' paper on backpropagation learning and borrowed the idea from Soviet mathematician Boris Polyak's 1964 article on solving functional equations. Stochastic gradient descent with momentum remembers the update Δw at each iteration, and determines the next update as a linear combination of the gradient and the previous update:

Momentum optimization -- Part 11

Stochastic gradient descent (often abbreviated SGD) is an iterative method for optimizing an objective function with suitable smoothness properties (e.g. differentiable or subdifferentiable). It can be regarded as a stochastic approximation of gradient descent optimization, since it replaces the actual gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data). Especially in high-dimensional optimization problems this reduces the very high computational burden, achieving faster iterations in exchange for a lower convergence rate. The basic idea behind stochastic approximation can be traced back to the Robbins–Monro algorithm of the 1950s. Today, stochastic gradient descent has become an important optimization method in machine learning.

Momentum optimization -- Part 12

Each $\{G(i,i)\}$ gives rise to a scaling factor for the learning rate that applies to a single parameter w_i . Since the denominator in this factor, $G_i = \sum_{\tau=1}^t g_{\tau}^2$ $\{\text{textstyle } \sqrt{G_{\{i\}}}\} = \sqrt{\sum_{\tau=1}^t g_{\tau}^2}$ is the L_2 norm of previous derivatives, extreme parameter updates get dampened, while parameters that get few or small updates receive higher learning rates. While designed for convex problems, AdaGrad has been successfully applied to non-convex optimization.

Momentum optimization -- Part 13

where, γ $\{\text{displaystyle } \gamma\}$ is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,